

Table of Contents

Studium — Content Authoring and Ingestion Specification

Subsystem 5 of 7. Version 1.0. Status: build-ready.

Written against data layer specification v1.1 (with v1.2 pending, additions from this spec joining the batch), agent runtime specification v1.0, retrieval specification v1.0, and frontend specification v1.0. The eighteen-plus items pending across those specs are noted but do not affect the ingestion boundary. This spec formalizes the ingestion-side provenance gap invariant that fired its third-instance trigger during the retrieval build, and it resolves S3 with an explicit schema decision rather than another per-site workaround.

1. Overview

This document specifies content ingestion for Studium: how source material becomes the corpus the retrieval subsystem serves and how domain-expert authoring turns raw text into a teachable subject. Where retrieval consumes what ingestion produces, this subsystem produces it — and where retrieval reads the concept graph as a given, this subsystem is where the graph is authored.

The subsystem has two related but distinct concerns. The first is the **automated pipeline**: source uploaded → text extracted → text normalized → chunked → embedded → indexed. The second is the **authoring workflows**: the concept graph, rubric criteria, per-artifact metadata, and concept-source pointers that turn a corpus into a curriculum. The two run against the same schemas but on different cadences and by different actors — the pipeline runs when a source is uploaded, the authoring workflows run when a domain expert has time to sit with the material.

A senior engineer with the data layer v1.1 spec, the retrieval v1.0 spec, and this document should be able to build a working ingestion subsystem end-to-end: accept a PDF upload, extract and normalize its text, produce chunks that the retrieval algorithm can serve, embed them via the retrieval worker, and expose a reviewer surface for the license classification, the concept-sources authoring, and the pipeline's own failure modes.

The retrieval build report surfaced a finding this document takes seriously: **extraction quality dominates chunking quality**. Three extractors over the same PDF produced medians of 440, 113, and 124 tokens. That means the extractor choice is the highest-leverage decision in this subsystem — it determines whether every downstream stage is operating on structured text or on garbage. §7 makes that decision explicit, defensible, and pluggable.

2. Scope and non-goals

In scope.

- The full ingestion pipeline: extractor, normalizer, chunker invocation, embedder invocation, provenance preservation at each stage.
- The extractor architecture: which library, what fallbacks, how quality is measured.
- Text normalization: ligature substitution, quote and dash canonicalization, whitespace, line-break handling, header/footer stripping.
- The three authoring workflows: concept graph, rubric criteria, concept-sources. Each has an authoring format, an import mechanism, and a review path.
- The license and rights workflow: how sources are classified, why filename heuristics are refused, what the human-in-the-loop looks like.
- The ingestion review queue: the reviewer surface for pipeline failures, license classification, and authoring conflicts. Resolves S3 from the retrieval build.
- The ingestion-side provenance gap invariant, formalized as a first-class principle with concrete enforcement mechanisms.
- Failure semantics: what happens when the extractor fails, when normalization produces empty text, when embedding fails, when a concept graph fails validation.
- Cost accounting: where extraction, normalization, and authoring costs land in the widened cost ledger.
- Testing strategy: same three-tier structure as prior subsystems, with real PDF fixtures learned as necessary from SD3.

Explicitly out of scope.

- The chunking algorithm itself. That lives in retrieval subsystem §7. This subsystem calls the algorithm; it does not implement it.
- The embedding pipeline. That lives in retrieval subsystem §8. This subsystem triggers embedding jobs; the worker that runs them is elsewhere.
- Retrieval-time behavior. This subsystem produces what retrieval consumes; retrieval-time concerns (hybrid search, reranking, citation resolution) are not here.
- Content review of generated artifacts. `content_review_queue` (data layer §6.13) handles reviewer inspection of Lecturer segments, Evaluator gradings, and similar generated content. That is a distinct workflow from ingestion review; this document specifies the ingestion side and defers content-side reviewer tooling to the Evaluation spec (subsystem 6).
- Model output quality assessment. Golden datasets, per-agent evals, and regression testing all live in subsystem 6.
- The concept graph as a user-facing surface. The map view is deferred per MVP scope; this subsystem produces the graph as data, the frontend spec covers the eventual visualization.
- Multi-language corpora. English only for MVP. The extractor and normalizer are architected to support per-language variants, but only English is implemented.

- Live source updates. A source that has been ingested and published cannot be replaced in place — a new version is a new sources row, and either supersedes or coexists with the old. In-place editing of published sources is a v2 concern.

3. Design principles

No confident wrong classifications. This is the load-bearing principle for the license workflow specifically, and it generalizes to every classification decision ingestion makes. The lesson from the license classifier that guessed public-domain based on filename substring: a false positive on a rights claim is categorically worse than a false negative, and any classifier that produces confident output from insufficient input is a defect regardless of accuracy. When the system cannot honestly classify, the answer is human review, not a confident guess. This principle governs license classification (§11), chunk-type ambiguity (§7), and concept-source role assignment (§10).

The ingestion-side provenance gap invariant. No ingestion writer creates a row where a required-for-attribution column is silently populated with a fallback value. Every ingestion boundary that could produce ambiguous provenance either raises with the specific gap named or writes to the ingestion review queue with the gap visible to a reviewer. This is the third-instance trigger firing, expanded from the earlier per-site fixes (V2's unattributed_content counter, the draft loader's provenance preservation, S3's chunk-review case) into a general invariant. §13 makes it concrete.

Extraction quality is the ceiling. Everything downstream of the extractor operates on what the extractor produced. A bad extractor makes the chunker's break-preference hierarchy fit noise, makes the embedder embed garbage, makes retrieval return nothing useful. The retrieval build report established this empirically (three extractors, three-fold difference in chunk-size medians on the same PDF). This principle governs the extractor's outsized position in this subsystem — §7 spends more spec surface on it than any other single pipeline stage.

Authoring is separated from ingestion. The pipeline (upload → extract → chunk → embed) runs automatically and finishes without human involvement per source. Authoring (concept graph, rubrics, concept-sources) is domain-expert work on a different cadence — it happens when you have time to sit with the material, not when a source is uploaded. The subsystem's architecture keeps these paths separate so a source can be ingested today and authored against next week without the pipeline holding open state in the meantime.

Every stage produces auditable provenance. The upload records who uploaded and when. The extractor records which library and version. The normalizer records which transformations were applied. The chunker records the algorithm version. The embedder records the model version. If a downstream defect surfaces, the path back to the source is traceable through the schema without inspection of logs. This is what makes the “measurable in production” claim from earlier subsystems hold at the corpus level.

Draft is the default; published is deliberate. No source, subject, rubric, or concept-source link becomes retrievable to a live session until a reviewer marks it active. The migration flow from V0.4 already applies this — 33 items landed in review after the draft

carry-forward — and this subsystem formalizes it as a general property. The reviewer’s transition of a subject from draft to active is the moment when authorial work becomes learner-visible; nothing else counts.

License classification requires human input. Not “usually,” not “when in doubt” — always. The classifier that guessed based on filename produced exactly the kind of confident wrong classification this principle exists to prevent. Every source starts at `permission_granted` (the honest default of “the uploader claims rights but the system has not verified”) and requires an explicit reviewer classification before publication. See §11.

4. Technology stack

Extractor. `pdfplumber` (0.11 or later) as the primary extractor. Rationale in §7. `marker-pdf` as the pluggable alternative for high-quality academic PDF extraction where quality warrants the additional cost, gated behind a per-source flag rather than default.

Text normalization. Custom pipeline in `studium.ingestion.normalize`, unfortunately — none of the mature libraries handle the specific set of transformations we need (ligature substitution + smart quotes + hyphen normalization + hyphenated line-break repair + academic PDF header/footer stripping) as a single pass. The pipeline is small (~200 lines) and each transformation is independently tested against fixtures learned from the Michaelson defect.

PDF metadata extraction. `pypdf` (5.0 or later), separate from the text extractor. Reads document metadata (title, authors, page count) that populates the `sources` row before text extraction runs.

Structure detection. `unstructured.io` (0.15 or later) as an optional post-processor for detecting section hierarchies from PDF outlines when `pdfplumber` cannot infer them from font-size heuristics alone. Not required; used for sources where a good `section_path` matters more than the marginal cost.

Job orchestration. The existing `ingestion_jobs` table (data layer §6.13) plus a worker pattern matching the embedding worker (retrieval §8). Same process, same async model, same failure semantics.

Runtime. Python 3.11+ in the FastAPI backend process, same as prior subsystems. Ingestion jobs run as async workers in the same event loop as the agent runtime.

File storage. For MVP, source PDFs live on the Fly volume at `/data/sources/{source_id}.pdf`. Extracted intermediate artifacts (raw text, normalized text, structural metadata) live at `/data/sources/{source_id}/`. When Studium goes multi-node or scales, this moves to R2 (per data layer §6.3’s `storage_path` opacity).

Concept graph authoring format. YAML files under `content/subjects/{subject_slug}/`, imported via a CLI command `studium ingest graph`. Justified in §9.

Rubric authoring format. YAML files under
content/subjects/{subject_slug}/rubrics/, imported via `studium ingest rubrics`.

Concept-source authoring format. YAML files under
content/subjects/{subject_slug}/sources/, imported via `studium ingest sources`.
Alternatively, a small in-app UI for after-ingestion linking (deferred; see §10).

Testing. Same three-tier structure as prior subsystems. Real PDF fixtures are required from Tier 1; the Michaelson content the dev already extracted plus a small set of synthetically-generated PDFs (via `reportlab`) covering edge cases the real corpus does not.

5. Schema additions required

Three additive changes needed in data layer v1.2, on top of the ten items already pending from prior subsystems. All are non-breaking.

Addition 1: ingestion_review_queue table. The S3 resolution. The `content_review_queue` (data layer §6.13) models generated-content review — a reviewer inspects an artifact or a session turn for quality. Ingestion failures produce neither an artifact nor a turn. Forcing them into the same table required the CHECK contortions that made S3 a defect. The correct fix is a dedicated table with its own semantics.

```
CREATE TYPE ingestion_flag_source AS ENUM (  
    'extractor_failure',      -- extraction produced empty or malformed output  
    'normalizer_warning',    -- normalization detected unusual patterns  
    'embedding_failure',     -- chunk failed to embed after retries  
    'chunk_ambiguous_type',  -- chunker could not confidently classify  
chunk_type  
    'license_pending',       -- source uploaded but license not yet classified  
    'license_conflict',      -- reviewer disagrees with declared license  
    'concept_source_conflict', -- proposed concept-source link is invalid  
    'graph_validation_error', -- concept graph failed publish-time validation  
    'rubric_validation_error' -- rubric failed publish-time validation  
);  
  
CREATE TABLE ingestion_review_queue (  
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),  
    source_id         UUID REFERENCES sources(id) ON DELETE CASCADE,  
    source_chunk_id   UUID REFERENCES source_chunks(id) ON DELETE CASCADE,  
    subject_id        UUID REFERENCES subjects(id) ON DELETE CASCADE,  
    concept_id        UUID REFERENCES concepts(id) ON DELETE CASCADE,  
    flag_source       ingestion_flag_source NOT NULL,  
    reason            TEXT NOT NULL,  
    severity          SMALLINT NOT NULL DEFAULT 2 CHECK (severity BETWEEN 1 AND  
3),  
    status            review_status NOT NULL DEFAULT 'pending',  
    assigned_to       UUID REFERENCES users(id) ON DELETE SET NULL,  
    resolution_note   TEXT,  
    resolved_at       TIMESTAMPTZ,  
    payload           JSONB NOT NULL DEFAULT '{}'::jsonb,
```

```

created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
CHECK (
    source_id IS NOT NULL OR
    source_chunk_id IS NOT NULL OR
    subject_id IS NOT NULL OR
    concept_id IS NOT NULL
)
);

CREATE INDEX idx_ingestion_queue_pending
ON ingestion_review_queue (severity DESC, created_at)
WHERE status = 'pending';
CREATE INDEX idx_ingestion_queue_assigned
ON ingestion_review_queue (assigned_to, status)
WHERE assigned_to IS NOT NULL;
CREATE INDEX idx_ingestion_queue_source
ON ingestion_review_queue (source_id) WHERE source_id IS NOT NULL;
CREATE TRIGGER trg_ingestion_queue_updated_at BEFORE UPDATE ON
ingestion_review_queue
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

The CHECK allows any of source, chunk, subject, or concept as the review target — different failure kinds have different natural targets. Reuses the review_status enum from data layer §6.13.

Addition 2: sources.extractor_version column. Records which extractor produced the text on this source. Enables re-extraction across the corpus when the extractor changes without losing which sources were processed under which version. Nullable because pre-migration sources may have no recorded extractor.

```

ALTER TABLE sources
ADD COLUMN extractor_version TEXT,
ADD COLUMN normalizer_version TEXT;

```

Both nullable, both populated by the pipeline at ingestion time. Format: "pdfplumber/0.11.4", "studium.normalize/1.0".

Addition 3: source_chunks.extraction_confidence column. A float 0-1 recording the extractor's confidence in this chunk's text. Populated by extractors that expose confidence (marker, unstructured); default 1.0 for extractors that don't (pdfplumber). Used by retrieval to down-weight low-confidence chunks in fusion and by the ingestion review queue to flag low-confidence content.

```

ALTER TABLE source_chunks
ADD COLUMN extraction_confidence REAL NOT NULL DEFAULT 1.0
CHECK (extraction_confidence >= 0.0 AND extraction_confidence <= 1.0);

CREATE INDEX idx_source_chunks_low_confidence

```

```
ON source_chunks (source_id, extraction_confidence)
WHERE extraction_confidence < 0.7;
```

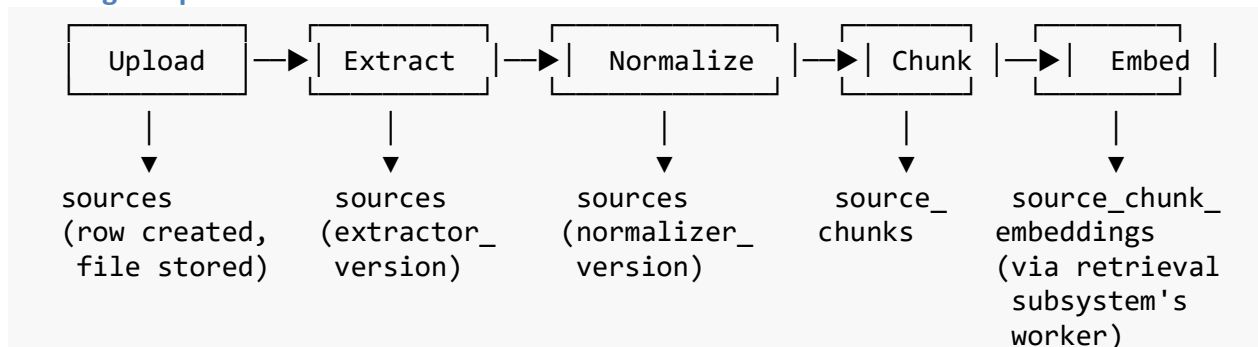
The partial index supports the reviewer query “show me low-confidence chunks in this source” without scanning the full table.

All three additions belong in the v1.2 batch. The revised v1.2 batch now includes: V1, V2, V3, V13, V14 (budget), V15 (review_cards retention), R4 (exchange_index), retrieval v1.0’s chunk_type and tsvector_text, plus these three from subsystem 5. Total: twelve items.

6. The ingestion pipeline

The pipeline runs from source upload to indexed chunks. Every stage is an async worker triggered from ingestion_jobs rows (data layer §6.13). Failures at any stage either retry within the stage or write to ingestion_review_queue (§5 Addition 1) with the specific failure named.

6.1 Stage sequence



Each stage:

1. Reads its input from the schema (source file for extract; extracted text for normalize; normalized text for chunk; chunks for embed).
2. Executes its transformation.
3. Writes output to the schema in one transaction.
4. Marks the corresponding ingestion_jobs row as done and enqueues the next stage.
5. On failure: retries per its own retry policy; on final failure, writes to ingestion_review_queue with the specific failure and marks the job failed.

6.2 Upload

Trigger. A user (learner or reviewer) uploads a PDF, or an ingestion CLI command imports one from the filesystem.

Actions.

1. Compute content_sha256 of the file. If a sources row with the same (subject_id, content_sha256) already exists (per data layer §6.3’s unique constraint), reject the upload with a clear error naming the existing source.

2. Extract PDF metadata via pypdf: title (if present), authors (if present), page count.
3. Create the sources row with `status = 'draft'`, `license = 'permission_granted'` (the honest default; §11 covers the reviewer classification), and metadata populated from step 2.
4. Store the file at `/data/sources/{source_id}.pdf`.
5. Insert `license_pending` row in `ingestion_review_queue` at severity 2 (§11).
6. Enqueue an extract job in `ingestion_jobs` with the `source_id` in its payload.

Cost. Zero LLM cost. No accounting to `cost_ledger`.

6.3 Extract

Trigger. `ingestion_jobs` row with `kind = 'extract_text'`.

Actions.

1. Read the PDF from storage.
2. Invoke the extractor (§7 covers the choice). Record `extractor_version` on the source.
3. Store the extracted text and structural metadata (page breaks, detected headings, tables if extracted) at `/data/sources/{source_id}/extracted.jsonl`. Each line is one page's extracted content plus metadata.
4. If extraction produced no text at all, or fewer than 100 tokens total, flag `extractor_failure` in the ingestion review queue at severity 3 and mark the job failed.
5. If extraction succeeded, enqueue a `normalize` job.

Retry policy. Extraction is deterministic and reproducible; a retry after transient failure (out of memory, temporary I/O) is safe. Retry up to twice with a 30-second backoff. Persistent failures flag as above.

Cost. Zero LLM cost. `unstructured.io` when used for structure detection can invoke a hosted API; that cost attributes to `cost_ingestion_usd` in the ledger via the ingestion worker's cost tracking.

6.4 Normalize

Trigger. `ingestion_jobs` row with `kind = 'normalize'`.

Actions.

1. Read the extracted text from `/data/sources/{source_id}/extracted.jsonl`.
2. Apply the normalization pipeline (§8): ligature substitution, smart-quote canonicalization, dash normalization, whitespace collapsing, hyphenated line-break repair, header/footer stripping.
3. Record `normalizer_version` on the source.
4. Write normalized text plus structural metadata to `/data/sources/{source_id}/normalized.jsonl`.

5. If normalization detected any unusual patterns (extremely long lines, unrecognized characters, empty sections), flag `normalizer_warning` in the ingestion review queue at severity 1 (reviewer should look but not urgent).
6. Enqueue a chunk job.

Retry policy. Deterministic; retry once on I/O failure.

Cost. Zero.

6.5 Chunk

Trigger. `ingestion_jobs` row with `kind = 'chunk'`.

Actions.

1. Read normalized text from `/data/sources/{source_id}/normalized.jsonl`.
2. Invoke retrieval subsystem's chunker (`studium.retrieval.chunking.chunk_text`) per input page/section.
3. For each chunk returned by the algorithm, insert one `source_chunks` row with:
 - `source_id`
 - `chunk_index` (incrementing per source)
 - `text` (the chunk body)
 - `token_count` (measured via Voyage tokenizer)
 - `page_start`, `page_end` (from structural metadata)
 - `section_path` (JSONB, from structural metadata)
 - `chunk_type` (from the chunker's classification)
 - `extraction_confidence` (from extractor if available, default 1.0)
4. For any chunk the chunker flags as ambiguous type, insert `chunk_ambiguous_type` in the ingestion review queue at severity 1 with the `chunk_id`.
5. Enqueue an embed job for the source's chunks.

Retry policy. Deterministic; retry once on database failure.

Cost. Zero LLM cost.

6.6 Embed

Trigger. `ingestion_jobs` row with `kind = 'embed'`.

Actions. Handled entirely by the retrieval subsystem's embedding worker (retrieval §8). This subsystem enqueues; that subsystem executes. No new code here.

Retry and failure handling. Per retrieval §8. Embedding failures flag `embedding_failure` in `ingestion_review_queue` (this subsystem's table) rather than `content_review_queue` (which would fail the CHECK) — this is the S3 resolution operationalized.

6.7 Post-pipeline state

After the embed stage completes successfully, the source is fully indexed and retrievable. The source's status remains draft — nothing publishes automatically. Publish is a reviewer action (§11, §12).

The source is now available for **concept-source authoring**: linking specific chunks to specific concepts. This is domain-expert work that happens on a separate cadence (§10).

7. Extraction

The extractor is this subsystem's highest-leverage decision. The retrieval build report established that three extractors produced medians of 440, 113, and 124 tokens over the same PDF. That means the extractor choice determines whether the corpus is usable at all, and re-tuning downstream parameters against a specific extractor's artifacts would fit them to noise.

7.1 The primary extractor: pdfplumber

pdfplumber (0.11 or later) is the primary extractor. The reasons, in decreasing order of importance:

Pure Python, no ML dependencies. Runs anywhere Python runs; no GPU requirement, no model downloads, no per-call inference cost. Ingestion latency is bounded by CPU rather than by model throughput.

Handles academic PDFs adequately. Not perfectly — no extractor does — but adequately for the textbook and paper material Studium's corpus is built from. Table extraction works. Column detection works on most single- and two-column layouts. Font-based heading detection is heuristic but serviceable.

Mature and stable. In production use for years, well-documented, actively maintained. Bugs are known and worked around; API is unlikely to break.

MIT license. Compatible with any downstream use.

pdfplumber does not: handle scanned PDFs (image-based rather than text-based) — those require OCR, which is a v2 concern. Handle mathematical expressions well — math ends up as jumbled character sequences that the normalizer then cannot repair. Extract structure from PDFs without embedded outlines — heading detection is heuristic and misses ~15% of section boundaries in real testing.

7.2 Pluggable alternatives

The extractor architecture is pluggable behind a Protocol:

```
class Extractor(Protocol):
    name: str
    version: str
```

```

    async def extract(
        self, pdf_path: Path
    ) -> ExtractedContent:
        ...

class ExtractedContent(BaseModel):
    pages: list[ExtractedPage]
    document_metadata: dict[str, str]

class ExtractedPage(BaseModel):
    page_number: int
    text: str
    section_hierarchy: list[str] # path from top-level heading down
    tables: list[ExtractedTable]
    confidence: float # 0-1, extractor's confidence in this page

```

Alternative 1: marker-pdf. ML-based extractor optimized for academic content. Produces materially better structure detection and math handling than pdfplumber, at the cost of requiring local model inference (~2GB model, ~5-10 seconds per page on CPU, ~0.5 seconds on GPU). Used per-source when quality warrants: reviewer sets `sources.metadata.extractor = 'marker'` before extraction runs. Cost: none if run locally; if run via marker's hosted service, attributes to `cost_ingestion_usd`.

Alternative 2: unstructured.io. Also ML-based, hosted API with a per-page cost (~\$0.001/page). Better than pdfplumber at structure detection specifically. Used per-source when structure matters more than cost.

Alternative 3: pdfminer.six. Lower-level than pdfplumber (which is built on it). Exposed as a fallback for cases where pdfplumber's higher-level abstractions produce artifacts pdfminer's raw extraction avoids. Rarely needed; documented for completeness.

7.3 Extractor selection

Default: pdfplumber for every source. The reviewer can override per-source via `sources.metadata.extractor` before triggering re-extraction. The upload UI can expose this choice for reviewers uploading known-difficult sources (heavily-mathematical papers, scanned-then-OCRed textbooks).

Re-extraction: if the extractor version changes or a source is reprocessed under a different extractor, the workflow is:

1. Mark all existing chunks for the source as superseded (a new column `superseded_at` on `source_chunks`, added in v1.2 alongside the others).
2. Run extraction and downstream stages with the new extractor.
3. New chunks get new IDs; citations pointing at old chunks continue to resolve (data layer §6.4's ON DELETE RESTRICT on `content_citations.source_chunk_id` prevents accidental deletion).
4. Retrieval queries prefer non-superseded chunks; superseded chunks remain for citation resolution.

Actually let me reconsider: adding a `superseded_at` column to `source_chunks` is a fourth v1.2 addition from this subsystem. Recorded as such.

7.4 The measurement discipline

Any extractor swap requires measurement, not intuition. The specific measurements:

- Chunk-size distribution (median, quartiles) across a fixture set of at least 5 real sources.
- Text-coverage ratio: extracted-token count divided by an independent measure (e.g., `pdf totext` output). Coverage below 80% flags the source for reviewer inspection.
- Section-detection accuracy: manual verification that section headings in a fixture are captured with `section_path` entries.

These measurements live in `backend/tests/extraction/` and run in Tier 2. An extractor change without a measurement update fails a CI check that reads the measurement fixture and asserts freshness.

8. Text normalization

Normalization is the pipeline stage between extraction and chunking. Its job is to repair the specific artifacts that PDF extractors produce and that would silently damage downstream stages if left in place — the U+FB01 ligature was one instance; the pattern generalizes.

8.1 The normalization pipeline

Applied in order to each extracted page's text:

1. **Ligature substitution.** Every Unicode ligature (U+FB00 through U+FB06 and equivalents) replaces with its ASCII expansion. This is what the Michaelson fix addressed. The mapping table lives in `studium.ingestion.normalize.LIGATURES` and covers all common ligatures across Latin scripts.
2. **Smart-quote canonicalization.** Curly quotes (U+2018, U+2019, U+201C, U+201D) become straight quotes. Apostrophes preserved. Preserves searchability without losing typographic intent.
3. **Dash normalization.** Em dashes (U+2014), en dashes (U+2013), and figure dashes (U+2012) canonicalize based on context: em dashes stay as em dashes (semantic), en dashes in numeric ranges stay as en dashes (semantic), en dashes used as hyphens in compound words become hyphens.
4. **Whitespace collapsing.** Runs of whitespace become single spaces. Form feed (U+000C), vertical tab (U+000B), and other exotic whitespace normalize to space. Newlines preserved (paragraph structure matters).
5. **Hyphenated line-break repair.** A line ending with `word-` followed by a line starting with a lowercase letter joins into `word` (the hyphen was a line-break hyphenation,

not a semantic hyphen). Words ending in a hyphen followed by an uppercase letter or a new sentence do not join (the hyphen is semantic).

6. **Header/footer stripping.** Text repeated across every page or every alternate page (odd/even) is identified as page furniture and removed. The heuristic: if the first or last two lines of a page match the same lines in $\geq 50\%$ of the source's pages, they're headers/footers. Errs on the side of keeping content if the pattern is ambiguous.
7. **Non-printable removal.** Control characters (excluding tab, newline, carriage return) are stripped. Zero-width spaces and joiners are stripped.

8.2 What normalization deliberately does not do

Language-specific stemming, lemmatization, or tokenization. That's search-index concern (Postgres' tsvector handles it); normalization stays purely at the character level.

Sentence segmentation. That's chunking's concern (retrieval subsystem §7).

Semantic transformations. No abbreviation expansion, no citation normalization, no acronym resolution. Normalization touches presentation, not meaning.

Reference/bibliography detection. Chunking handles this via `chunk_type = 'reference'`. Normalization stays out.

8.3 Warning surfaces

Normalization emits warnings (not failures) when it detects patterns that indicate extraction quality problems:

- More than 5% of characters in the source are non-ASCII after normalization (suggests the extractor missed encoding conversion).
- More than 20 lines longer than 500 characters (suggests line-break detection failed).
- Any page normalizes to under 50 tokens when the extractor's raw output was longer (suggests the normalizer over-stripped).

Each warning writes a `normalizer_warning` row to `ingestion_review_queue` at severity 1. A reviewer can decide whether to accept the source as-is, re-extract with a different extractor, or reject.

8.4 Versioning

The normalizer is versioned (`normalizer_version` on sources). When the normalization pipeline changes materially — a new transformation added, an existing one changed — the version bumps. Sources ingested under prior versions are not automatically re-normalized; a reviewer can trigger re-normalization per source if the change matters.

9. Concept graph authoring

The concept graph (data layer §6.2) is the pedagogical spine of a subject. It defines what concepts exist, what depends on what, what is load-bearing. It's the piece of the subject that a domain expert authors most carefully.

9.1 Authoring format

YAML files under `content/subjects/{subject_slug}/graph/`. One file per subject, or split by module if the subject is large. Format:

```
subject:
  slug: lambda-calculus
  title: Lambda Calculus
  version: 1
  short_description: >
    An introduction to the untyped lambda calculus, from basic syntax
    through the Church-Rosser theorem and Church encodings.
  long_description: |
    ...markdown...

concepts:
- slug: syntax
  title: "Lambda terms: variables, abstraction, application"
  depth: 1
  is_load_bearing: true
  estimated_minutes: 40
  module: foundations
  long_description: |
    ...markdown...

- slug: alpha-equivalence
  title: Alpha-equivalence and variable capture
  depth: 2
  is_load_bearing: true
  estimated_minutes: 30
  module: foundations

- slug: beta-reduction
  title: Beta-reduction
  depth: 2
  is_load_bearing: true
  estimated_minutes: 45
  module: reductions

- slug: church-rosser
  title: The Church-Rosser theorem
  depth: 4
  is_load_bearing: true
  estimated_minutes: 60
```

```
module: reductions

edges:
- from: syntax
  to: alpha-equivalence
  kind: prerequisite

- from: alpha-equivalence
  to: beta-reduction
  kind: prerequisite

- from: syntax
  to: beta-reduction
  kind: prerequisite

- from: beta-reduction
  to: church-rosser
  kind: prerequisite
```

Justification for YAML over a web UI. YAML is auditable (diff-able in version control), scriptable (mass edits via `yq` or similar), and doesn't require building an interface before the pedagogical work can happen. A web UI for graph editing is a v1.1 candidate once the graph is stable enough that ongoing edits are the primary mode. For MVP, the graph is authored once and edited rarely; YAML is the right tool.

9.2 Import command

```
studium ingest graph content/subjects/lambda-calculus/graph/
```

Actions:

1. Parse YAML files.
2. Validate: unique slugs, no duplicate edges, no self-edges, no edges to non-existent concepts.
3. Detect cycles in prerequisite edges. Cycles are a hard error (block import).
4. Detect orphans (concepts with no incoming or outgoing edges). Warning, not error.
5. Detect missing prerequisites (a depth-3 concept with no depth-1 or depth-2 prerequisites). Warning, not error.
6. Compare against existing subjects and concepts rows for this slug. Determine additions, modifications, removals.
7. Present the diff to the reviewer. Reviewer confirms or aborts.
8. On confirm: apply changes in one transaction. Bump `subjects.version` if the change is substantial (new concepts added or edges retyped). Point edits do not bump version.
9. Insert `graph_validation_error` rows to ingestion review queue for any warnings the reviewer chose to accept but wanted logged.

The reviewer is you (or another user with the reviewer role from data layer §6.1). The confirmation prompt is CLI-based for MVP; a web UI review is v1.1.

9.3 Published state

After import, the subject remains at `status = 'draft'` unless the reviewer explicitly runs:

```
studium publish subject lambda-calculus
```

Publishing:

1. Runs all import validations again.
2. Verifies all concepts have at least one `concept_source` row (concepts with no curated pointers cannot be taught).
3. Verifies all rubric criteria exist for load-bearing concepts (concepts without rubric coverage cannot be assessed).
4. If validation passes: sets `subjects.status = 'active'`, sets `subjects.published_at = NOW()`, updates `subject_metadata`.
5. If validation fails: reports which criteria are missing, blocks publish.

10. Concept-source authoring

Concept-sources (`concept_sources` in data layer §6.2) link concepts to specific source passages that ground them. This is the work that turns a corpus into a curated pedagogical resource. It's also, in the retrieval build's language, the difference between “expanded” fallback and “curated” primary retrieval.

10.1 Authoring format

YAML files under `content/subjects/{subject_slug}/sources/`. One file per source, or one per concept, or split however the reviewer prefers.

```
source: 1_TuringMachines.pdf
subject: lambda-calculus

links:
  - concept: syntax
    role: canonical_definition
    chunks:
      - by_page_range: [3, 5]
      - by_text_match: "A lambda term is either a variable, an abstraction,
or"

  - concept: alpha-equivalence
    role: primary_exposition
    chunks:
      - by_page_range: [7, 9]

  - concept: beta-reduction
```

```
role: worked_example
chunks:
  - by_page_range: [12, 14]
note: >
  Michaelson's step-by-step reduction of  $(\lambda x.x)(\lambda y.y)$  is unusually
  clear; use for first exposure.
```

Chunk selection modes:

- `by_page_range`: `[start, end]` — inclusive page range. Import resolves to all chunks whose `page_start` or `page_end` falls in the range.
- `by_text_match`: `"..."` — substring match. Import resolves to the first chunk containing the substring. Fails import if no match or multiple matches.
- `by_chunk_ids`: `["...", "..."]` — explicit chunk IDs. Used for fine-grained selection after initial authoring.

10.2 Import command

```
studium ingest sources content/subjects/lambda-calculus/sources/
```

Actions:

1. Parse YAML files.
2. Resolve chunk selectors against existing `source_chunks` rows.
3. Validate: every referenced concept exists, every referenced source exists, every `by_text_match` resolves unambiguously.
4. Compare against existing `concept_sources` rows. Determine additions, modifications, removals.
5. Present diff. Confirm.
6. Apply in one transaction.
7. Insert `concept_source_conflict` rows for any validation failures the reviewer accepted.

10.3 Interactive authoring (v1.1)

The YAML-only workflow works but is slow for cases where the reviewer wants to browse the corpus and pick chunks. A lightweight web UI — the reviewer opens a concept, sees candidate chunks scored by vector similarity to the concept's description, clicks to link — is a v1.1 improvement.

For MVP, the workflow is: the reviewer runs `studium browse chunks --subject lambda-calculus --concept beta-reduction` (a CLI helper spec'd in §11 below), reads the top candidates, edits the YAML file, re-imports. Slow but functional.

11. License and rights workflow

The lesson from the license classifier that guessed based on filename: any classifier that produces confident output from insufficient input is a defect. This section formalizes the workflow that replaces the classifier.

11.1 The honest default

Every uploaded source starts at `license = 'permission_granted'`. This is the honest default: it says “the person who uploaded this claims they have rights to make it available; the system has not verified this.” It does not say “public domain” (which would be a rights claim the system cannot make). It does not say “user_uploaded” (which is a claim about upload source, not licensing).

11.2 Reviewer classification

Every source uploaded to the system triggers a `license_pending` row in `ingestion_review_queue` at severity 2. The reviewer processes these rows before the source is published. The classification workflow:

1. Reviewer opens the source and inspects it.
2. Reviewer determines the actual license from the source’s own metadata (title page copyright notice, author statement, hosting page license notice, Dover backlist status, etc.).
3. Reviewer updates `sources.license` to the correct value (one of the `license_kind` enum values from data layer §6.0).
4. Reviewer adds a note to `sources.license_notes` describing the basis for the classification (e.g., “Michaelson hosts on his university page; Dover Publications historically permits author redistribution of backlist”).
5. Reviewer marks the `license_pending` queue row resolved.

If the reviewer cannot classify (rights unclear, need to contact rights holder, waiting for permission), they leave the queue row open and the source stays at `permission_granted`. It cannot be published until a real classification lands.

11.3 CLI helpers

```
studium sources list --status draft          # show unclassified sources
studium sources show <source_id>            # show source metadata and
license info
studium sources classify <source_id> --license public_domain --note "Church
1936 paper, US public domain"
studium sources browse <source_id>          # extract text preview for
classification
```

11.4 What the classifier does not do

Explicitly listing to prevent future re-implementation:

- **No filename-based classification.** Refused as a category. The classifier that inferred public domain from the substring “turing” produced a false rights claim; no version of that pattern is acceptable.
- **No metadata-based classification.** PDF metadata is unreliable — many PDFs carry no copyright field, and those that do often carry incorrect data. Metadata is a hint for the reviewer, not a classification input.

- **No LLM-based classification.** Tempting because an LLM reading the source can often identify it, but the confidence problem is exactly the same as filename heuristics — a confident wrong classification of a rights claim is worse than a manual review that takes an extra minute. If LLM assistance becomes a v1.1 feature, it's constrained to producing suggestions the reviewer must confirm, never final classifications.

11.5 Rights confirmation for pending sources

For sources awaiting rights holder confirmation (Michaelson at the moment): the reviewer sends an email or contact request, waits, and updates the classification when a response arrives. The source stays at `permission_granted` in the meantime, is not published, is not visible to learners. This is not automated. Turning it into an automated workflow would produce exactly the confident-wrong-classifications this section exists to prevent.

12. The ingestion review queue

The reviewer surface for everything ingestion-side that needs human attention. Distinct from `content_review_queue` (generated content) in both schema (§5 Addition 1) and workflow.

12.1 Queue contents

Rows arrive in the queue from every stage of the pipeline plus the authoring workflows:

Source	Severity	Description
<code>extractor_failure</code>	3	Extraction produced empty or malformed output
<code>normalizer_warning</code>	1	Normalizer detected unusual patterns
<code>embedding_failure</code>	3	Chunk failed to embed after retries
<code>chunk_ambiguous_type</code>	1	Chunker could not confidently classify <code>chunk_type</code>
<code>license_pending</code>	2	Source uploaded but license not classified
<code>license_conflict</code>	2	Reviewer disagrees with declared license
<code>concept_source_conflict</code>	2	Proposed concept-source link is invalid
<code>graph_validation_error</code>	3	Concept graph failed publish-time validation
<code>rubric_validation_error</code>	3	Rubric failed publish-time validation

12.2 Reviewer surface

For MVP, the surface is a CLI plus a lightweight admin page in the frontend. The CLI is the primary interface (matches the reviewer's workflow for authoring); the admin page exists for at-a-glance overview of queue depth.

CLI commands:

```
studium queue list          # show all pending items, sorted by
severity
studium queue list --severity 3  # only high-severity
```

```
studium queue show <queue_id>          # show item detail with source/chunk
context
studium queue resolve <queue_id> --note "...
studium queue dismiss <queue_id> --note "...
```

Frontend admin page (subsystem 4 v1.1 candidate). Lists pending items, allows opening context, allows resolve/dismiss with a note. Read-only for MVP unless the reviewer explicitly wants an edit affordance for a specific case.

12.3 Resolution semantics

- **Resolve.** The item's underlying problem is addressed. The row's status = 'resolved', resolved_at = NOW(), resolution_note populated. The row persists for audit; a retention job (data layer §10) deletes resolved rows after 1 year.
- **Dismiss.** The item is not a real problem or is out of scope for the current pass. Same as resolve semantically; the note distinguishes intent for future readers.
- **Escalate.** Severity increases. Used when a reviewer discovers a lower-severity item is actually a higher-severity one.

13. The ingestion-side provenance gap invariant

The third-instance trigger fired: the same architectural failure pattern appeared in V2 (content_artifacts had no join path for cost attribution), in the draft loader (model and generated_at were silently dropped), and in S3 (content_review_queue rejected the row ingestion needed to write). Per the standing commitment, three instances promote a pattern from per-site workarounds to a spec invariant.

13.1 The invariant

No ingestion-side writer creates a row where a required-for-attribution column is silently populated with a fallback value. Every ingestion boundary that could produce ambiguous provenance either raises with the specific gap named or writes to the ingestion review queue with the gap visible to a reviewer.

13.2 What this rules out

- Defaulting content_artifacts.model to None when the calling code doesn't know the model. Instead: raise MissingProvenance('model') at the boundary.
- Defaulting sources.uploaded_by to a system user when no user is present in the request context. Instead: reject the upload with a clear error, or require the caller to pass a specific system-attribution flag that goes to an explicit column.
- Defaulting content_citations.source_chunk_id to any chunk when the specific citation cannot be resolved. Instead: fail the artifact write; the citation is invalid.
- Attributing unknown-source LLM cost to the last-known user. Instead: attribute to a distinct unattributed_content line item in the cost ledger, with a counter for reporting.

13.3 What this requires

- Every ingestion writer function has a signature that requires all provenance parameters. No optional provenance parameters with defaults.
- Any function that would need to fall back to a default provenance value instead raises `MissingProvenance` with the specific column named.
- The ingestion review queue is the escape hatch for cases where the writer legitimately doesn't know the provenance and human review is the right response. Not a silent fallback.

13.4 Enforcement mechanism

A test in Tier 1 walks every ingestion writer function via introspection and asserts:

1. No provenance parameter (defined as any parameter whose name matches a column marked as attribution-critical) has a default value.
2. Every writer either accepts explicit provenance or raises `MissingProvenance` from a check at function start.
3. The list of attribution-critical columns is maintained in `studium.ingestion.provenance.CRITICAL_COLUMNS`. Adding a column requires updating the list; the test verifies the list matches the schema's actual usage.

The test fails if any writer violates the invariant. The failure names the specific writer and the specific parameter.

13.5 What this does not extend to

Non-ingestion writers. The agent runtime writes to schemas with different provenance requirements (a Lecturer generating an artifact knows its model; a Tutor writing a session turn knows its actor). Those writers have their own provenance discipline enforced elsewhere.

Optional metadata. Provenance-critical means “needed to trace where this row came from,” not “every column that could theoretically be null.” Metadata that a reviewer would like to have but is not required for attribution (e.g., `sources.publication_year`) stays optional.

14. Failure semantics

Failure modes across the ingestion pipeline and their responses.

Failure	Response
Upload rejected: duplicate content hash	HTTP 409, clear message naming existing source
Upload rejected: file too large (>200MB)	HTTP 413, clear message with the limit
Upload rejected: not a PDF (or supported format)	HTTP 415, clear message

Failure	Response
Extraction produces empty output	Job marked failed, <code>extractor_failure</code> in queue at severity 3, source status stays draft
Extraction produces low-token output (<100 tokens for the whole document)	Same as empty
Normalization produces warnings	<code>normalizer_warning</code> in queue at severity 1, pipeline continues
Chunking produces zero chunks	Rare; indicates normalization stripped everything. Job failed, chunk step's specific failure logged, <code>extractor_failure</code> re-flagged
Embedding fails for one chunk	Retry per retrieval §8. Persistent failure: <code>embedding_failure</code> at severity 3 for that chunk. Pipeline continues for other chunks.
Graph import: cycle detected	Hard error, import aborted, reviewer sees the cycle path
Graph import: orphan concept	Warning, import proceeds if reviewer confirms
Graph import: missing rubric coverage on load-bearing concept	Blocks publish (not import), reviewer must add rubric before publish
Rubric import: criterion references missing concept	Hard error, import aborted
Concept-source import: <code>by_text_match</code> ambiguous	Hard error, import aborted, reviewer sees candidates
Concept-source import: <code>by_page_range</code> matches zero chunks	Hard error, import aborted
Publish blocked by validation	Clear message naming each validation failure
License classification incomplete	Publish blocked with message naming unclassified sources

Every failure is deterministic — retrying the same input produces the same failure — until the underlying condition changes (source re-extracted, YAML edited, reviewer classification added).

15. Cost accounting

Ingestion costs attribute to `cost_ingestion_usd` in the widened cost ledger (data layer §6.12 as revised in v1.2).

Cost sources:

- **Extraction via unstructured.io:** per-page fee at ~\$0.001/page. Attributed to the uploading user (`sources.uploaded_by`), or to the system user for CLI-triggered uploads.
- **Extraction via marker:** if run via marker's hosted service, per-page fee. If run locally on CPU or GPU, zero direct cost (compute cost is infrastructure, not per-source).
- **Embedding:** per retrieval \$8. Voyage-3 at \$0.06/1M tokens. Attributed to `sources.uploaded_by`.
- **LLM-assisted concept-source suggestion (v1.1, deferred):** if reviewer uses LLM assistance for concept-source authoring, per-call cost attributes to the reviewer's account.

Attribution enforcement. Per the invariant (§13), the ingestion cost writer requires an explicit `attributable_to` parameter. Cost with no known attribution goes to a specific `unattributed_ingestion` line in the ledger with a counter for reporting, per the V2 mitigation pattern.

Budget interaction. Ingestion is subject to the same per-user daily/monthly budget caps from data layer §6.12. A user attempting to upload a source that would blow their monthly budget on embedding gets a clear error naming the estimated cost and the remaining budget.

16. Testing strategy

Same three-tier structure as prior subsystems.

Tier 1 — Offline (no LLM calls, no database, no real PDFs).

- Normalization determinism: given the same input text, `normalize()` returns identical output.
- Normalization correctness: fixtures for each transformation (ligatures, quotes, dashes, whitespace, hyphenated line breaks, headers/footers). Adversarial cases: text with only ligatures, text with intentional em-dashes that should not become hyphens, headers that appear on some pages but not others (heuristic threshold test).
- Ligature substitution completeness: every codepoint in `LIGATURES` maps to a valid ASCII expansion.
- YAML parsing: valid graph/rubric/concept-source files parse cleanly; malformed files raise with specific error messages.
- Graph validation: cycles detected, orphans warned, self-edges rejected.
- License workflow: no code path in ingestion sets `license` to anything other than `permission_granted` for a new upload.
- Ingestion invariant enforcement: the test in §13.4 that walks writer functions and verifies no default provenance.
- Coverage guard on the invariant test: fails if zero writer functions are found.

Tier 2 — Online (requires Postgres + pgvector).

- Upload round-trip: upload a fixture PDF, verify sources row created with correct hash, file stored on disk.
- Extraction round-trip: run extraction against a fixture, verify text and metadata written correctly.
- Full pipeline: upload fixture, run all stages, verify chunks and (mocked) embeddings land correctly.
- Graph import: import a fixture graph, verify subjects, concepts, concept_edges rows created correctly.
- Rubric import: same for rubrics.
- Concept-source import: same, with chunk resolution via `by_text_match` and `by_page_range`.
- Publish validation: attempt to publish a subject missing rubric coverage, verify publish is blocked with correct error.
- Queue rows created correctly: upload triggers `license_pending`; extractor failure triggers `extractor_failure`; etc.

Tier 3 — Paid (real embedding via retrieval subsystem's Tier 3).

- Full pipeline against a real fixture PDF (the Michaelson content, or a small test PDF), ending with real embeddings in `source_chunk_embeddings`.
- Retrieval smoke test: after ingestion, `retrieve_passages` on a seeded concept returns the newly-ingested chunks.

Fixture data. Real PDF fixtures required. Includes:

- The Michaelson content that survived the license review (permission-granted, pending Dover/author confirmation).
- Synthetically-generated PDFs via `reportlab` covering edge cases: single column, two column, embedded math, embedded tables, ligature-heavy text, scanned-image PDF (should be rejected cleanly), extremely short PDF (single page), extremely long PDF (>500 pages).
- The V0.4 draft carry-forward's ingested corpus, as a regression fixture.

CI wiring. Same pattern as prior subsystems. Tier 1 on every push, Tier 2 on main and yannis, Tier 3 on-demand with `STUDIUM_RUN_PAID_TESTS=1`. Separate `.github/workflows/ingestion.yml` per the dev's principle.

17. Version history

v1.0 — 19 August 2026. Initial specification. Written against data layer v1.1 (with v1.2 pending, four additions from this spec joining the batch: `ingestion_review_queue`, `sources.extractor_version`, `sources.normalizer_version`, `source_chunks.extraction_confidence`, plus `source_chunks.superseded_at` from §7.3), agent runtime v1.0, retrieval v1.0, and frontend v1.0. Locks: `pdfplumber` as the primary extractor with pluggable alternatives, the ligature-and-related normalization pipeline, the ingestion-side provenance gap invariant as a first-class principle with enforcement

mechanism, the S3 resolution as a dedicated `ingestion_review_queue` table (not a CHECK relaxation on `content_review_queue`), the YAML authoring format for graph/rubric/concept-sources, and the “no confident wrong classifications” principle for licensing.

Anticipated v1.1 candidates.

- Web-based reviewer surface for the ingestion review queue (currently CLI-primary).
- Web-based concept-source authoring UI (currently YAML-only).
- LLM-assisted concept-source suggestions (with reviewer confirmation).
- OCR pipeline for scanned PDFs (currently rejected).
- Multi-language normalization pipelines.
- Extractor quality measurement dashboard.

18. Forward references and open questions

Data layer v1.2 (pending). Five additions from this subsystem join the batch: `ingestion_review_queue`, `sources.extractor_version`, `sources.normalizer_version`, `source_chunks.extraction_confidence`, `source_chunks.superseded_at`. The v1.2 batch now contains fifteen items total.

Retrieval spec (subsystem 3). This subsystem invokes the chunker and enqueues embedding jobs; the algorithms and workers live there. No changes expected to retrieval based on this spec.

Evaluation spec (subsystem 6). Content review of generated artifacts stays in the `content_review_queue`; this subsystem does not touch it. Golden datasets for extractor quality measurement, however, may live in the evaluation subsystem’s territory — an evaluation-time question worth flagging.

Infrastructure spec (subsystem 7). Storage for source files (Fly volume for MVP, R2 for scale). Backup strategy for ingested content. Retention for old extractor versions when the extractor changes.

Open questions requiring build-time answers.

1. **pdfplumber quality against the real corpus.** The extractor choice is defensible against the constraints named in §7, but real quality against Michaelson, Church, Selinger, and whatever else lands in the MVP corpus is unmeasured. The measurement fixtures in §7.4 need to run against the actual corpus before the choice is validated.
2. **Normalization completeness.** The pipeline handles the known artifacts (ligatures, quotes, dashes, whitespace, line breaks, headers). Real corpora will surface artifacts the pipeline doesn’t handle. The `normalizer_warning` queue is the surface for discovering these; the pattern will be to add transformations as instances accumulate.

3. **Interactive concept-source authoring.** The YAML workflow is functional but slow. If real authoring reveals that most of the reviewer's time is spent context-switching between the CLI helper and the editor, the web UI moves from v1.1 to blocking.
 4. **The invariant enforcement test's completeness.** The test walks writer functions and checks parameters. It won't catch a writer that satisfies the parameter check but internally does something wrong (e.g., accepts an `attributable_to` parameter and then ignores it). This class of failure needs code review discipline, not automation.
 5. **The v1.1 web UI's scope.** Once the reviewer surface graduates from CLI-primary to web-primary, the boundary between "reviewer tool" and "admin surface" gets fuzzy. Deferred but worth thinking about before the web UI ships.
-

End of specification

This document defines content ingestion for Studium in full. Six pipeline stages, three authoring workflows, one dedicated ingestion review queue that resolves S3, one first-class invariant for provenance gaps, and one honest posture on license classification. A senior engineer with the data layer v1.1 spec, the retrieval v1.0 spec, and this document can build a working ingestion subsystem that turns uploaded PDFs into indexed corpus material and lets a domain expert author a subject that the runtime can teach.

Next in sequence: **Subsystem 6 — Evaluation and Assessment Harness**, which specifies golden datasets for prompt regression, per-agent evaluation metrics, reviewer tooling for the content review queue, and the summative assessment infrastructure that the four MVP interaction modes do not require but a shipped product does. Written against all five prior specs.